From Timing Protection to Structural Change: Robust Synchronization in One-Van, One-Robot Delivery

Abstract

This paper presents a compact mixed-integer linear programming model for a last-mile delivery system with one van and one robot. The van departs from a start depot, serves a subset of customers directly, and ends at a return depot. Customers not visited by the van may instead be served by the robot through sorties attached to single van arcs. The principal modeling issue is synchronization under uncertainty: robot outbound travel time, robot service time, and robot inbound travel time are subject to bounded upward deviations. To protect synchronization, the model incorporates waiting on van arcs and penalizes that waiting in the objective.

The computational study follows a completed four-stage benchmark. Week 1 solves a deterministic baseline on small exact instances. Week 2 subjects that deterministic benchmark to a Monte Carlo stress test under robot-time uncertainty while keeping the deterministic policy fixed. Week 3 solves a robust synchronization model and compares it with the deterministic benchmark at $\alpha = 0.10$. Week 4 expands that comparison across $\alpha \in \{0.05, 0.10, 0.20\}$. All experiments are conducted in a one-van, one-robot setting on exact small-instance models with $n \in \{5, 8, 10\}$, seeds $\{1, 2, 42\}$, and an AMPL + Gurobi notebook implementation.

The results show that the deterministic benchmark solves successfully but is highly fragile under uncertainty: across the tested grid, the deterministic failure rate remains 1.0 under the Monte Carlo reliability test. The robust synchronization model restores reliability, reducing failure rate to 0.0 in all tested instances. At low and medium uncertainty levels, robustness acts primarily as timing protection, preserving route and sortie structure while increasing waiting. At the highest tested uncertainty level, however, structural change begins to emerge. In two key cases, $(5, 0.20, 1)$ and $(5, 0.20, 42)$, the robust model changes the route, the sortie pattern, or both. The main conclusion is that waiting-based robustness is effective as a reliability safeguard, but its operational role shifts from timing protection to possible structural redesign as uncertainty intensifies.

1 Problem Description

We consider a last-mile delivery system consisting of one van and one robot. The van starts at node 0, visits a subset of customers directly, and terminates at node $n + 1$. Customers not visited by the van may instead be served by the robot. A robot sortie is associated with a single van arc: the robot launches from node k after van service at k, serves a customer i, and is recovered at the next van node ℓ while the van travels directly from k to ℓ.

The central modeling challenge is synchronization under uncertainty. Robot outbound travel time, robot service time, and robot inbound travel time may all exceed their nominal values. The formulation addresses this issue by allowing waiting on van arcs and penalizing waiting in the objective.

The completed computational study follows a four-stage structure:

Week 1: deterministic baseline,

Week 2: deterministic stress test under uncertainty,

Week 3: robust comparison at $\alpha = 0.10$,

Week 4: alpha-level expansion across $\{0.05, 0.10, 0.20\}$.

This progression is designed to separate nominal planning performance from robustness effects and to identify the point at which robustness begins to affect service structure.

2 Modeling Assumptions

The formulation relies on the following assumptions:

1. There is one van and one robot.
2. The van starts at node 0 and ends at node $n + 1$.
3. Each customer is served exactly once, either by the van or by the robot.
4. A robot sortie is attached to exactly one van arc (k, ℓ).
5. If sortie (k, i, ℓ) is selected, the robot launches from node k after van service at k, serves customer i, and is recovered at node ℓ.
6. At most one robot sortie may be assigned to any used van arc.
7. Robot endurance is enforced by preprocessing.
8. Uncertainty is represented through bounded upward deviations.
9. Van travel times and van service times are deterministic.
10. Uncertainty affects only robot outbound travel, robot service time, and robot inbound travel.

3 Notation

3.1 Sets and Indices

- $C = \{1, \ldots, n\}$: set of customers,
- $V = \{0\} \cup C \cup \{n+1\}$: set of van nodes,
- 0: start depot,
- $n+1$: end depot,
- A: feasible van arc set,
- F^e: candidate robot sortie set before endurance preprocessing,
- F: feasible robot sortie set after preprocessing,
- i, j: customer indices,
- k, ℓ: van-node indices.

The feasible van arc set is

$$A = \{(k, \ell) \in V \times V : k \neq \ell,\ \ell \neq 0,\ k \neq n+1\}. \tag{1}$$

The candidate robot sortie set is

$$F^e = \{(k, i, \ell) : (k, \ell) \in A,\ i \in C,\ i \neq k,\ i \neq \ell\}. \tag{2}$$

3.2 Parameters

- $c^V_{k\ell}$: van travel cost on arc (k, ℓ),
- $\tau^V_{k\ell}$: van travel time on arc (k, ℓ),
- s^V_k: van service time at node k,
- τ^R_{ki}: nominal robot outbound travel time from k to i,
- s^R_i: nominal robot service time at customer i,
- $\tau^R_{i\ell}$: nominal robot inbound travel time from i to ℓ,
- $c^R_{ki\ell}$: robot sortie cost for sortie (k, i, ℓ),
- $\delta^{R,+}_{ki}$: maximum upward deviation in outbound robot travel time,
- $\delta^{S,+}_i$: maximum upward deviation in robot service time,

- $\delta_{i\ell}^{R,+}$: maximum upward deviation in inbound robot travel time,
- E_R: robot endurance,
- T_k: valid upper bound on van arrival time at node k,
- H: route horizon,
- $\gamma > 0$: waiting penalty coefficient,
- $\lambda > 0$: route-completion penalty coefficient,
- $W_{k\ell}$: upper bound on waiting on arc (k, ℓ),
- $M_{ki\ell}$: big-M value for synchronization waiting.

We set

$$s_0^V = s_{n+1}^V = 0. \tag{3}$$

To distinguish the deterministic and robust models clearly, we define two sortie-duration quantities.

Deterministic nominal sortie duration. For each $(k, i, \ell) \in F^e$,

$$\bar{\rho}_{ki\ell} = \tau_{ki}^R + s_i^R + \tau_{i\ell}^R. \tag{4}$$

Robust worst-case sortie duration. For each $(k, i, \ell) \in F^e$,

$$\rho_{ki\ell} = \tau_{ki}^R + s_i^R + \tau_{i\ell}^R + \delta_{ki}^{R,+} + \delta_i^{S,+} + \delta_{i\ell}^{R,+}. \tag{5}$$

The deterministic feasible sortie set is

$$F^{\text{det}} = \{(k, i, \ell) \in F^e : \bar{\rho}_{ki\ell} \leq E_R\}, \tag{6}$$

and the robust feasible sortie set is

$$F^{\text{rob}} = \{(k, i, \ell) \in F^e : \rho_{ki\ell} \leq E_R\}. \tag{7}$$

For each $(k, \ell) \in A$, define the deterministic waiting upper bound

$$W_{k\ell}^{\text{det}} = \max\left\{0, \max_{i:(k,i,\ell)\in F^{\text{det}}} \left(\bar{\rho}_{ki\ell} - \tau_{k\ell}^V\right)\right\}, \tag{8}$$

and the robust waiting upper bound

$$W_{k\ell}^{\text{rob}} = \max\left\{0, \max_{i:(k,i,\ell)\in F^{\text{rob}}} \left(\rho_{ki\ell} - \tau_{k\ell}^V\right)\right\}. \tag{9}$$

For each $(k, i, \ell) \in F^{\text{det}}$, define

$$M_{ki\ell}^{\text{det}} = \max\left\{0,\ \bar{\rho}_{ki\ell} - \tau_{k\ell}^{V}\right\}, \tag{10}$$

and for each $(k, i, \ell) \in F^{\text{rob}}$, define

$$M_{ki\ell}^{\text{rob}} = \max\left\{0,\ \rho_{ki\ell} - \tau_{k\ell}^{V}\right\}. \tag{11}$$

3.3 Decision Variables

- $x_{k\ell} \in \{0, 1\}$: equals 1 if the van traverses arc (k, ℓ),
- $y_{ki\ell} \in \{0, 1\}$: equals 1 if the robot launches from k, serves i, and recovers at ℓ while the van traverses arc (k, ℓ),
- $t_k \geq 0$: van arrival time at node k,
- $w_{k\ell} \geq 0$: waiting time on van arc (k, ℓ),
- $f_{k\ell} \geq 0$: single-commodity flow variable.

4 Compact MILP Formulation

The deterministic and robust models share the same MILP structure. They differ only in the sortie-duration parameter, feasible sortie set, waiting upper bound, and big-M values employed.

For compactness, we present the formulation using generic symbols F, $\tilde{\rho}_{ki\ell}$, $W_{k\ell}$, and $M_{ki\ell}$. In the deterministic model,

$$F = F^{\text{det}}, \qquad \tilde{\rho}_{ki\ell} = \bar{\rho}_{ki\ell}, \qquad W_{k\ell} = W_{k\ell}^{\text{det}}, \qquad M_{ki\ell} = M_{ki\ell}^{\text{det}},$$

where $\bar{\rho}_{ki\ell}$ is defined in (4). In the robust model,

$$F = F^{\text{rob}}, \qquad \tilde{\rho}_{ki\ell} = \rho_{ki\ell}, \qquad W_{k\ell} = W_{k\ell}^{\text{rob}}, \qquad M_{ki\ell} = M_{ki\ell}^{\text{rob}},$$

where $\rho_{ki\ell}$ is defined in (5).

4.1 Objective Function

$$\min \sum_{(k,\ell)\in A} c_{k\ell}^{V} x_{k\ell} + \sum_{(k,i,\ell)\in F} c_{ki\ell}^{R} y_{ki\ell} + \gamma \sum_{(k,\ell)\in A} w_{k\ell} + \lambda t_{n+1}. \tag{12}$$

4.2 Service and Routing Constraints

Each customer is served exactly once:

$$\sum_{k:(k,i)\in A} x_{ki} + \sum_{(k,i,\ell)\in F} y_{ki\ell} = 1 \qquad \forall i \in C. \tag{13}$$

The van leaves the start depot once:

$$\sum_{\ell:(0,\ell)\in A} x_{0\ell} = 1. \tag{14}$$

The van arrives at the end depot once:

$$\sum_{k:(k,n+1)\in A} x_{k,n+1} = 1. \tag{15}$$

Flow conservation at each customer:

$$\sum_{k:(k,i)\in A} x_{ki} = \sum_{\ell:(i,\ell)\in A} x_{i\ell} \qquad \forall i \in C. \tag{16}$$

At most one sortie may be attached to any used van arc:

$$\sum_{i:(k,i,\ell)\in F} y_{ki\ell} \le x_{k\ell} \qquad \forall (k,\ell) \in A. \tag{17}$$

4.3 Connectivity Constraints

Depot flow:

$$\sum_{\ell:(0,\ell)\in A} f_{0\ell} = \sum_{i\in C} \sum_{k:(k,i)\in A} x_{ki}. \tag{18}$$

Customer flow balance:

$$\sum_{k:(k,i)\in A} f_{ki} - \sum_{\ell:(i,\ell)\in A} f_{i\ell} = \sum_{k:(k,i)\in A} x_{ki} \qquad \forall i \in C. \tag{19}$$

Arc capacities:

$$0 \le f_{k\ell} \le n x_{k\ell} \qquad \forall (k,\ell) \in A. \tag{20}$$

4.4 Time and Horizon Constraints

Start time:

$$t_0 = 0. \tag{21}$$

Arrival upper bounds:

$$0 \le t_k \le T_k \qquad \forall k \in V. \tag{22}$$

If customer i is not visited by the van, then t_i is forced to zero:

$$t_i \le T_i \sum_{k:(k,i)\in A} x_{ki} \qquad \forall i \in C. \tag{23}$$

Route horizon:

$$t_{n+1} \le H. \tag{24}$$

4.5 Synchronization Constraints

If the van traverses arc (k, ℓ), then

$$x_{k\ell} = 1 \;\Rightarrow\; t_\ell \ge t_k + s_k^V + \tau_{k\ell}^V + w_{k\ell} \qquad \forall (k,\ell) \in A. \tag{25}$$

In the implemented notebook workflow, this constraint is enforced through AMPL indicator syntax and solved with Gurobi.

The synchronization waiting constraint is

$$w_{k\ell} \ge \tilde{\rho}_{ki\ell} - \tau_{k\ell}^V - M_{ki\ell}(1 - y_{ki\ell}) \qquad \forall (k,i,\ell) \in F. \tag{26}$$

Waiting is allowed only on used arcs:

$$0 \le w_{k\ell} \le W_{k\ell} x_{k\ell} \qquad \forall (k,\ell) \in A. \tag{27}$$

4.6 Variable Domains

$$x_{k\ell} \in \{0,1\} \qquad \forall (k,\ell) \in A, \tag{28}$$

$$y_{ki\ell} \in \{0,1\} \qquad \forall (k,i,\ell) \in F, \tag{29}$$

$$f_{k\ell} \ge 0 \qquad \forall (k,\ell) \in A, \tag{30}$$

$$t_k \ge 0 \qquad \forall k \in V, \tag{31}$$

$$w_{k\ell} \ge 0 \qquad \forall (k,\ell) \in A. \tag{32}$$

5 Validity of the Robust Synchronization Constraint

This section establishes that the robust synchronization layer guarantees robot recovery feasibility for every admissible realization in the uncertainty set.

5.1 Uncertainty Set

For each robust-feasible sortie $(k, i, \ell) \in F^{\text{rob}}$, suppose

$$\tau_{ki}^{R,\omega} \in [\tau_{ki}^{R},\ \tau_{ki}^{R} + \delta_{ki}^{R,+}], \tag{33}$$

$$s_{i}^{R,\omega} \in [s_{i}^{R},\ s_{i}^{R} + \delta_{i}^{S,+}], \tag{34}$$

$$\tau_{i\ell}^{R,\omega} \in [\tau_{i\ell}^{R},\ \tau_{i\ell}^{R} + \delta_{i\ell}^{R,+}]. \tag{35}$$

The worst-case sortie duration is given by (5). If sortie (k, i, ℓ) is selected, the realized robot completion time is

$$T_{ki\ell}^{R,\omega} = t_k + s_k^V + \tau_{ki}^{R,\omega} + s_i^{R,\omega} + \tau_{i\ell}^{R,\omega}. \tag{36}$$

Proposition 1. *Consider any robust-feasible sortie $(k, i, \ell) \in F^{\text{rob}}$. If the model enforces*

$$w_{k\ell} \geq \rho_{ki\ell} - \tau_{k\ell}^{V} - M_{ki\ell}^{\text{rob}}(1 - y_{ki\ell}), \qquad \forall (k, i, \ell) \in F^{\text{rob}}, \tag{37}$$

together with the van timing indicator

$$x_{k\ell} = 1 \Rightarrow t_\ell \geq t_k + s_k^V + \tau_{k\ell}^{V} + w_{k\ell}, \qquad \forall (k, \ell) \in A, \tag{38}$$

then for every selected sortie with $y_{ki\ell} = 1$, robot recovery at node ℓ is feasible for every admissible realization in (33)–(35). That is,

$$T_{ki\ell}^{R,\omega} \leq t_\ell \tag{39}$$

for all admissible realizations.

Proof. Fix a selected sortie $(k, i, \ell) \in F^{\text{rob}}$ with $y_{ki\ell} = 1$. Then (37) reduces to

$$w_{k\ell} \geq \rho_{ki\ell} - \tau_{k\ell}^{V}.$$

Because a selected sortie can be attached only to a used van arc, (38) implies

$$t_\ell \geq t_k + s_k^V + \tau_{k\ell}^{V} + w_{k\ell}.$$

Substituting the lower bound on $w_{k\ell}$ yields

$$t_\ell \geq t_k + s_k^V + \tau_{k\ell}^{V} + \rho_{ki\ell} - \tau_{k\ell}^{V} = t_k + s_k^V + \rho_{ki\ell}.$$

By definition of $\rho_{ki\ell}$,

$$\tau_{ki}^{R,\omega} + s_i^{R,\omega} + \tau_{i\ell}^{R,\omega} \leq \rho_{ki\ell}$$

for every admissible realization in the uncertainty set. Adding $t_k + s_k^V$ to both sides gives

$$T_{ki\ell}^{R,\omega} = t_k + s_k^V + \tau_{ki}^{R,\omega} + s_i^{R,\omega} + \tau_{i\ell}^{R,\omega} \leq t_k + s_k^V + \rho_{ki\ell} \leq t_\ell.$$

Hence robot recovery at node ℓ is feasible for all admissible realizations. $\square$

Corollary 1. *The robust feasible set is contained in the deterministic feasible set. Any solution satisfying robust synchronization also satisfies deterministic nominal synchronization.*

This follows because $\rho_{ki\ell} \geq \bar{\rho}_{ki\ell}$ for every sortie. Therefore, any waiting level sufficient to protect the worst-case duration is also sufficient to protect the nominal duration.

6 Computational Experiment Design

6.1 Objective of the Study

The completed study follows a four-stage benchmark:

1. deterministic baseline,
2. deterministic stress test under uncertainty,
3. robust comparison at $\alpha = 0.10$,
4. alpha-level expansion across $\{0.05, 0.10, 0.20\}$.

The objective is to determine whether robustness serves primarily as timing protection or whether stronger uncertainty begins to induce structural changes in the service plan.

6.2 Instance Structure

The completed experiments use:

- one van,
- one robot,
- exact small-instance MILP models,
- $n \in \{5, 8, 10\}$,
- seeds $\{1, 2, 42\}$,
- $\alpha \in \{0.05, 0.10, 0.20\}$.

6.3 Instance Generation

Instances are generated from Euclidean coordinates. The same generator and parameter settings are used throughout Weeks 1–4:

- area size = 120.0,

- van speed = 1.0,
- robot speed = 1.3,
- van cost per distance = 1.0,
- robot cost per distance = 0.4,
- van service time = 3.0,
- robot service time = 1.0,
- robot endurance factor = 1.5,
- waiting penalty $\gamma = 0.10$,
- completion penalty $\lambda = 0.01$,
- horizon factor = 2.0.

6.4 Uncertainty Construction

Uncertainty is parameterized by

$$\alpha \in \{0.05, 0.10, 0.20\},$$

with

$$\delta_{ki}^{R,+} = \alpha \tau_{ki}^{R}, \tag{40}$$
$$\delta_{i}^{S,+} = \alpha s_{i}^{R}, \tag{41}$$
$$\delta_{i\ell}^{R,+} = \alpha \tau_{i\ell}^{R}. \tag{42}$$

6.5 Compared Methods

The completed study compares:

1. **Deterministic baseline:** nominal preprocessing and nominal synchronization,
2. **Robust synchronization model:** worst-case preprocessing and robust synchronization.

6.6 Monte Carlo Reliability Test

Reliability is assessed through Monte Carlo simulation. For each selected sortie (k, i, ℓ), each trial samples independently:

$$\tau_{ki}^{R,\omega} \in [\tau_{ki}^{R},\ \tau_{ki}^{R} + \delta_{ki}^{R,+}], \tag{43}$$
$$s_{i}^{R,\omega} \in [s_{i}^{R},\ s_{i}^{R} + \delta_{i}^{S,+}], \tag{44}$$
$$\tau_{i\ell}^{R,\omega} \in [\tau_{i\ell}^{R},\ \tau_{i\ell}^{R} + \delta_{i\ell}^{R,+}]. \tag{45}$$

The route, sortie, and waiting decisions are held fixed during simulation; there is no adaptive recourse. A synchronization violation is recorded if realized robot completion exceeds the planned van arrival time at the recovery node. Each simulation reports:

1. failure rate,
2. average number of synchronization violations,
3. average completion time under the fixed waiting policy.

The completed experiments use $N = 1000$ Monte Carlo trials per instance.

6.7 Implementation Workflow

The study is implemented in an AMPL + Gurobi notebook workflow in the NIU notebook environment:

- the MILP is written in AMPL,
- indicator constraints are modeled through AMPL and solved with Gurobi,
- Euclidean instances are generated programmatically,
- deterministic and robust models are solved through AMPL,
- simulation and CSV export are handled within the notebook workflow.

7 Results

7.1 Week 1: Deterministic Baseline

The deterministic compact van–robot model solved successfully on all tested small instances. The resulting solutions were nontrivial: each included robot sorties and positive waiting, indicating that the synchronization mechanism is active.

For the representative instance with $n = 5$, $\alpha = 0.10$, and seed 42, the deterministic solution selected route $[0, 4, 1, 6]$ with sorties $(0, 3, 4)$, $(1, 5, 6)$, and $(4, 2, 1)$, positive waiting on arcs $(0, 4)$, $(1, 6)$, and $(4, 1)$, objective value 236.988587, and completion time 262.819456.

Because the deterministic model depends only on nominal data, the deterministic benchmark is invariant with respect to α. Changing α affects uncertainty magnitudes, but not the nominal optimization inputs.

Table 1: Summary of Week 1 deterministic baseline results

n	Seeds	Feasible sorties $\lvert F \rvert$	Selected robot sorties
5	3	94–104	1–3
8	3	411–438	1–4
10	3	809–876	2–3

7.2 Week 2: Deterministic Stress Test Under Uncertainty

Week 2 evaluates the deterministic route, sortie, and waiting policy under sampled robot-time uncertainty while holding that policy fixed. No re-optimization, adaptive waiting, or recourse is permitted.

Across all tested instances and all three uncertainty levels, the deterministic benchmark is fully fragile:

- deterministic failure rate = 1.0 in every tested case,
- average synchronization violations equal the number of selected robot sorties,
- average completion time under uncertainty remains equal to planned completion time because the fixed waiting policy is replayed without delay propagation.

This behavior is consistent with a nominally tight synchronization design: once realized robot durations exceed nominal levels, the deterministic policy no longer protects recovery feasibility.

Table 2: Summary of Week 2 deterministic stress-test behavior

n	α levels	Deterministic failure rate	Avg. sync. violations
5	$0.05, 0.10, 0.20$	1.0 in all runs	equal to selected sorties
8	$0.05, 0.10, 0.20$	1.0 in all runs	equal to selected sorties
10	$0.05, 0.10, 0.20$	1.0 in all runs	equal to selected sorties

Representative cases include:

- $n = 5$, $\alpha = 0.10$, seed 42: three selected sorties, failure rate 1.0, average synchronization violations 3.0,
- $n = 8$, $\alpha = 0.10$, seed 2: four selected sorties, failure rate 1.0, average synchronization violations 4.0,
- $n = 10$, $\alpha = 0.10$, seed 42: two selected sorties, failure rate 1.0, average synchronization violations 2.0.

7.3 Week 3: Robust Comparison at $\alpha = 0.10$

Week 3 introduces the robust synchronization model and compares it directly with the deterministic benchmark at $\alpha = 0.10$ using the same Monte Carlo reliability test.

At this uncertainty level, the robust model restores reliability in every tested instance:

$$\text{deterministic failure rate} = 1.0, \qquad \text{robust failure rate} = 0.0.$$

Across all nine Week 3 runs,

- the van route remains unchanged,
- the selected robot sorties remain unchanged,
- the robust model increases waiting,
- completion time increases by the same amount as waiting,
- objective value increases modestly.

Accordingly, at $\alpha = 0.10$, robustness acts as *timing protection* rather than structural redesign.

Table 3: Summary of Week 3 comparison at $\alpha = 0.10$

n	Seeds	Same route	Same sorties	Det. fail. rate	Robust fail. rate
5	3	Yes	Yes	1.0 in all runs	0.0 in all runs
8	3	Yes	Yes	1.0 in all runs	0.0 in all runs
10	3	Yes	Yes	1.0 in all runs	0.0 in all runs

Representative cases include:

- $n = 5$, $\alpha = 0.10$, seed 42: same route and same three sorties; waiting gap 25.681945, completion-time gap 25.681946,
- $n = 8$, $\alpha = 0.10$, seed 2: same route and same four sorties; waiting gap 23.105158,
- $n = 10$, $\alpha = 0.10$, seed 42: same route and same two sorties; waiting gap 9.469396.

7.4 Week 4: Alpha-Level Expansion Across $\{0.05, 0.10, 0.20\}$

Week 4 extends the comparison to the full uncertainty grid

$$\alpha \in \{0.05, 0.10, 0.20\},$$

while keeping the same benchmark structure, instance sizes, and seeds.

The robust model remains fully reliable throughout the grid:

- average deterministic failure rate = 1.0 at every tested α,
- average robust failure rate = 0.0 at every tested α.

The central Week 4 question is therefore not whether robustness works, but how its function changes as uncertainty increases.

Table 4: Week 4 alpha-level summary

α	Det. failure	Robust failure	Avg. objective gap	Route same frac.	Sorties same frac.
0.05	1.0	0.0	0.909837	1.000000	1.000000
0.10	1.0	0.0	1.819674	1.000000	1.000000
0.20	1.0	0.0	4.603664	0.777778	0.777778

The timing-gap results suggest two operational regimes.

Low and medium uncertainty. At $\alpha = 0.05$, the average waiting gap and average completion-time gap are both 8.271246, and both route and sortie preservation fractions equal 1.0. At $\alpha = 0.10$, the same pattern persists: the average waiting gap and average completion-time gap are both 16.542491, and both preservation fractions again equal 1.0. In this range, robustness functions primarily as timing protection on an unchanged service structure.

High uncertainty. At $\alpha = 0.20$, the average waiting gap increases to 26.284751, whereas the average completion-time gap rises further to 34.022671. At the same time, the route preservation fraction falls to 0.777778, and the sortie preservation fraction also falls to 0.777778. High uncertainty is thus the first regime in which the robust model begins to alter service structure rather than merely adding timing slack.

Key structural-change cases. The clearest evidence of this transition comes from two instances: $(5, 0.20, 1)$ and $(5, 0.20, 42)$.

Table 5: Key structural-change cases at $\alpha = 0.20$

Case 1: $n = 5,\ \alpha = 0.20,$ **seed** $= 1$	
Deterministic plan	Route $[0, 2, 5, 1, 6]$ Sorties $(0, 4, 2),\ (2, 3, 5)$
Robust plan	Route $[0, 3, 2, 1, 6]$ Sorties $(1, 5, 6),\ (2, 4, 1)$
Gaps	Objective gap $= 11.639739$ Waiting gap $= 47.195645$ Completion-time gap $= 65.863387$
Case 2: $n = 5,\ \alpha = 0.20,$ **seed** $= 42$	
Deterministic plan	Route $[0, 4, 1, 6]$ Sorties $(0, 3, 4),\ (1, 5, 6),\ (4, 2, 1)$
Robust plan	Route $[0, 4, 1, 5, 6]$ Sorties $(0, 3, 4),\ (1, 2, 5)$
Gaps	Objective gap $= 7.152549$ Waiting gap $= -16.457285$ Completion-time gap $= 34.516259$

These cases are particularly informative. In the first case, the robust model changes both route and sortie structure while substantially increasing waiting. In the second, the robust model again changes both route and sorties, but total waiting decreases even though completion time increases and reliability is fully restored. The second case is especially revealing: at high uncertainty, robustness cannot be interpreted simply as added slack.

7.5 Integrated Discussion

Taken together, Weeks 1–4 establish a coherent progression:

- Week 1 shows that the deterministic compact formulation solves cleanly and produces coordinated plans.
- Week 2 shows that these deterministic plans are fragile under uncertainty.
- Week 3 shows that the robust synchronization layer restores reliability at $\alpha = 0.10$ while preserving route and sortie structure.
- Week 4 shows that this preservation pattern remains intact at low and medium uncertainty but may break at high uncertainty.

The completed study therefore identifies two regimes for robustness in the one-van, one-robot setting:

1. **Timing-protection regime:** at low and medium uncertainty, the robust model preserves the deterministic route and sortie design and primarily adds waiting.
2. **Structural-change regime:** at high uncertainty, the robust model may begin to alter the route, the sortie structure, or both.

This distinction is the central contribution of the alpha-level expansion. It clarifies not only that robustness restores reliability, but also how its operational role changes as uncertainty intensifies.

8 Limitations and Future Work

The present study is deliberately narrow. Its purpose is to isolate the effect of robust synchronization in a small exact benchmark before considering broader extensions.

8.1 Current limitations

1. **Single-van, single-robot setting.** The experiments consider one van and one robot only.
2. **Exact small-instance study only.** The completed study is restricted to $n \in \{5, 8, 10\}$, seeds $\{1, 2, 42\}$, and $\alpha \in \{0.05, 0.10, 0.20\}$.
3. **Fixed-policy simulation.** The Monte Carlo evaluation holds route, sortie, and waiting decisions fixed and does not include adaptive recourse or delay propagation.

8.2 Future work

Future work begins after this completed four-stage benchmark. The following directions were not part of the present study:

- heuristic methods,
- large-instance experiments,

- exact-versus-heuristic comparison,
- robot speed ratio sweeps,
- customer-density sweeps,
- endurance sweeps,
- richer uncertainty models,
- recourse or delay-propagation simulation,
- multi-van or multi-robot extensions.

Week 4 is therefore part of the completed study, not part of future work.

9 Conclusion

This paper developed and evaluated a four-stage benchmark for robust synchronization in a van–robot delivery setting:

1. deterministic baseline,
2. deterministic stress test under uncertainty,
3. robust comparison at $\alpha = 0.10$,
4. alpha-level expansion across $\{0.05, 0.10, 0.20\}$.

Several conclusions follow directly from the completed experiments.

First, the deterministic compact van–robot model solves cleanly and produces meaningful coordinated plans on the tested exact small instances.

Second, the deterministic benchmark is fragile under uncertainty. Across the full uncertainty grid, the deterministic failure rate remains 1.0, indicating that nominal synchronization is not reliable under upward robot-time variation.

Third, the robust synchronization model restores reliability. Across all tested uncertainty levels, the robust failure rate is 0.0 under the same Monte Carlo reliability test.

Most importantly, the alpha-level expansion shows that robustness has two operational regimes. At low and medium uncertainty, robustness acts primarily as timing protection: it preserves route and sortie structure while increasing waiting. At high uncertainty, robustness may induce structural change in the route, the sortie structure, or both.

The key evidence for this transition is provided by the two structural-change cases $(5, 0.20, 1)$ and $(5, 0.20, 42)$. These cases show that, once uncertainty becomes sufficiently large, restoring reliable synchronization may require more than additional waiting; it may require redesign of the service plan itself.

The main contribution of the paper is therefore both computational and structural. The waiting-based robust synchronization model restores reliability across the tested grid, and the completed four-stage benchmark clarifies when robustness acts as timing protection and when it begins to reshape service structure. Future work begins after this completed benchmark.

References

[1] Aharon Ben-Tal, Laurent El Ghaoui, and Arkadi Nemirovski. *Robust Optimization*. Princeton University Press, 2009.

[2] Dimitris Bertsimas and Melvyn Sim. The price of robustness. *Operations Research*, 52(1):35–53, 2004.

[3] Chase C. Murray and Amanda G. Chu. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C*, 54:86–109, 2015.

[4] Nils Boysen, Stefan Schwerdfeger, and Felix Weidinger. Scheduling last-mile deliveries with truck-based autonomous robots. *European Journal of Operational Research*, 271(3):1085–1099, 2018.

[5] James F. Campbell, David Sweeney, and Joseph Zhang. Strategic design for delivery with trucks and drones. *Transportation Research Part E*, 122:350–364, 2019.